Planning as Autoregressive Sequence Modelling: Decoding Strategies

Dian Kruger
Shane Josias

Introduction

Planning is a core problem in artificial intelligence: given a model of an environment and a desired objective, an agent must construct a sequence of actions that leads from an initial state to a goal state while satisfying constraints and optimising some notion of cost or reward. Classical approaches formulate planning as search over a state space (e.g., A*) [1], whereas reinforcement learning (RL) frames it as maximising cumulative reward through value functions or policies [2]. More recently, a generative modelling perspective has emerged, in which trajectories themselves are treated as structured sequences that can be modelled probabilistically. From this viewpoint, planning becomes a problem of inference in a learned distribution over trajectories rather than explicit search in a hand-designed state graph [3].

In this project, planning in a 2D gridworld is formulated as autoregressive sequence modelling. A trajectory is represented as an ordered sequence of states (or state–action pairs), and a model is trained using next-step prediction on a dataset of valid trajectories. [1]. Once trained, the model defines a learned probability distribution over feasible trajectories. Planning then corresponds to decoding: generating a trajectory from this distribution that satisfies the task objective. In this framing it is useful to distinguish between autoregressive generation and decoding. Autoregressive generation means that the model produces a trajectory sequentially, predicting each next state (or state–action token) conditioned on the previously generated tokens. Decoding refers to the rule used during inference to convert the model's predicted probability distribution into an actual trajectory. Decoding is necessary because multiple continuations are typically possible at each step, and a procedure is required to decide which one forms the final plan [3]. More precisely, The central question of this project is how different decoding strategies affect the quality, reliability, diversity, and computational cost of the generated plans. While the underlying model is fixed after training, inference procedures such as greedy decoding, stochastic sampling with temperature control, and beam search may extract substantially different trajectories from the same distribution [4]. Greedy decoding prioritises local likelihood, potentially sacrificing global optimality or diversity. Stochastic sampling can increase diversity but may introduce instability or suboptimal paths. Beam search balances exploration and exploitation at the cost of increased computation [4]. By systematically comparing these strategies, the project investigates how inference choices shape planning outcomes when planning is cast as generative sequence modelling.

Background

Generative Models

Generative models are types of machine learning systems that aim to represent a probability distribution over data, or to draw samples from that distribution [5, p. 493]. Specifically, if $\mathbf{x}$ is a collection of features observed from an object or event, like words in a sentence or coordinates in a maze, and $p(\mathbf{x})$ is the true joint distribution of those features, a generative model learns an approximation $\hat{p}(\mathbf{x})$ of this distribution from training observations [5].

Once such a distribution has been learned, it can be used to generate new observations that are similar to the observations in the training data by sampling from $\hat{p}(\mathbf{x})$ [5, p. 102].

In practice, assumptions are often made to simplify the learning problem, such as assuming a particular parametric form for the distribution (e.g., Gaussian), so instead of learning an arbitrary distribution, the model learns a specific family of distributions parameterised by θ. The learning process then involves finding the parameters θ that best fit the training data.

We thus assume that the data's true distribution $p(\mathbf{x})$ belongs to a family of distributions $\{p_\theta(\mathbf{x})\}$, and the model learns the parameters θ that maximise the likelihood of the training data under $p_\theta(\mathbf{x})$.

The Planning Problem

Broadly, automated planning is the field of research addressing the problem of finding an ordered or partially ordered series of actions that changes an environment from its initial state to a desired goal state [6].

In this work, the planning problem is to generate a valid trajectory from a given starting position to a specified goal position in a 2D Maze environment.

In this environment, a trajectory refers to a sequence of maze positions connecting a start location to a goal location without passing through walls. Assuming a discrete grid-based representation, maze coordinates are integers identifying grid cells. A trajectory can therefore be represented as a sequence of coordinates in which each consecutive pair corresponds to adjacent non-wall cells, with adjacency restricted to horizontal and vertical moves.

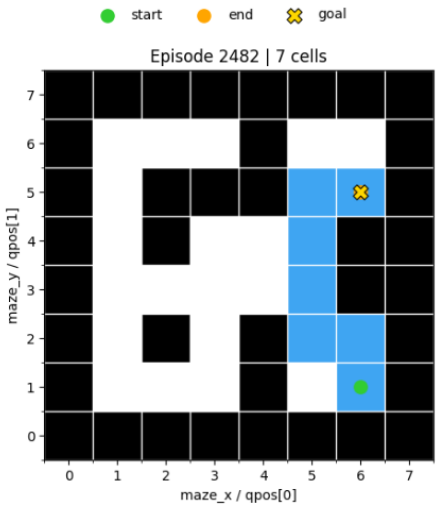

Figure 1: Example of a valid trajectory in a discrete 2D maze environment

Generative Modelling for Planning

If a dataset of valid trajectories is available, a generative model can be trained to learn the distribution over these trajectories. Once trained, the model can be used to generate new trajectories by sampling from the learned distribution. This approach treats planning as a problem of inference in a probabilistic model [3]. So if we define a trajectory τ with T steps as:

$$\tau = (s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$$

where s_t is the cordinates (states) at time step t and a_t is the action (moving up, down, left or right) taken at time step t, the generative model learns to approximate the distribution $p(\tau)$ over valid trajectories. Instead of directly modelling $p(\tau)$, an autoregressive model factorises this distribution into a product of conditional probabilities [5, p. 705]:

$$p(\tau) = \prod_{t=1}^{T} p(z_t \mid z_{<t})$$

where z_t denotes the tth token in the trajectory sequence, which may correspond to either a state or an action. This simplifies the supervised learning problem to modelling $p(z_t \mid z_{<t})$, so given a series of states and actions in a trajectory, estimate the probability of the next token in the sequence.

A complete trajectory is then generated autoregressively by repeatedly predicting the next token until a full plan is produced, with the decoding strategy determining how each next-token choice is made. The choice of decoding strategy is important because the model's predicted distribution may assign non-negligible probability mass to multiple valid continuations at each step, and the continuation selected during decoding can significantly affect both the quality of the final trajectory and the computational cost of generation [3]. Since the learned distribution is only an approximation to the true trajectory distribution, it may also assign non-negligible probability to invalid continuations. As a result, the decoding strategy can influence whether a valid trajectory is generated at all. Even when

multiple valid trajectories are possible, some may be more desirable than others, for example because they require fewer steps, and the decoding strategy can therefore affect whether a more optimal trajectory is produced [3, 4].

Transformer Architecture

The Transformer architecture is a type of neural network that has become the standard for sequence modelling tasks, notably in natural language processing. [4, p. 173] Even though the original Transformer was designed for language, the fact that it models sequences of tokens makes it applicable to our trajectory modelling problem, where the tokens could be states and/or actions in a trajectory. [3].

The key components of the Transformer architecture are now briefly introduced. For intuition, the discussion refers primarily to natural language processing examples, where the role of these components is often easier to interpret than in trajectory modelling.

1. Input Representation:

Before raw data enters the Transformer, it must be converted into a high dimensional vector representation.

- **Tokenisation**:

  The first step involves tokenisation, where the input sequence is broken down into discrete tokens. In an NLP scenario, this could invlolde splitting text into words, subwords or characters [4]. For the purposes of this discussion, it is assumed that the input has been tokenised at the word level.

- **Embeddings** (E): Each token ID is used to index into an embedding matrix E, which maps the discrete ID to a dense vector of dimensionality d. This matrix has shape $|V| \times d$, where $|V|$ is the vocabulary size, i.e. the total number of distinct tokens that the model can represent [4]. When tokens correspond to words, the embedding can be interpreted as capturing semantic and syntactic properties of those words. Intuitively, words with similar meanings, such as "big" and "large," may be represented by vectors that are close to one another in the embedding space [4].

- **Positional Encoding**: Since the Transformer architecture processes sequences in parrallel, it does not inherently encode the order of tokens, so a positional encoding is added to the token embeddings to ensure that the position of each token in the sequence is represented. This is done by adding a positional encoding vector to each token embedding [4].

- **Input Formation**: For a sequence of N tokens, the individual d-dimensional vector embeddings are placed into a single matrix X of size $N \times d$. This matrix serves as the input for the model [4].

2. Transformer Blocks:

Each block consits of a multi-head self-attention mechanism followed by a feedforward neural network, with each sublayer followed by a layer normalisation and a residual connection [4].

A usefull mental model is that everytime the input passes through a sublayer, the model adjusts the embeddings of the token to better capture the relevant information for the task at hand. For example, in an NLP context, the model may adjust the embedding of a word to better capture its meaning in the context of the sentence [4].

- **Layer Normalisation**:

  For training stability, layer normalisation is applied to ensure the values in the sublayers remain within a reasonable range.

  For a vector x, it is calculated as:

  $$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta$$

  where μ is the mean, σ the standard deviation of the vector's elements, and γ and β are learned parameters that scale and shift the normalized output. [4].

- **Multi-Head Self-Attention**:

  This is the core mechanism that allows the model incorporate information from all tokens in the sequence when processing each token [4]. Under the interpretation that each sub-layer progressively refines word representations, the self-attention mechanism allows the representation of each word to be updated in light of the representations of all other words in the sequence. For example, if the word "fly" is preceded by the word "the", the model may adjust its representation in a way that reflects the nominal sense of "fly" (the insect) rather than the verbal sense. Each input vector x_i is mapped into three learned representations: a Query (Q), a Key (K), and a Value (V).

  The attention for a single head is computed as:

  $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large and causing numerical instability. In causal (left-to-right) models, such as predicting the next word in a sentence, or the next position in a trajectory, a mask is applied to ensure that tokens can only attend to the past, preventing the model from "cheating" by looking at future tokens during training [4].

- **Feedforward Neural Network**:

  After the attention layer incorporates information across tokens, the Feedforward Network (FFN) processes each token position independently and in parallel. It is usually a two-layer network with a non-linear activation such as ReLU:

  $$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

This layer allows the model to perform more complex transformations of the contextualised information gathered by the attention heads.

Returning to the intuitive picture introduced earlier, the attention mechanism can be viewed as first updating each word representation using information from the other words in the sequence. The FFN then further refines each of these updated representations independently at each position, using transformations learned from many training sequences.

3. Prediction Head:

After the data passes through the final transformer block, the final representations are passed to the next-token prediction head, which converts them into a probability distribution over possible next tokens in the sequence.

- **Output Projection**: Let h_t denote the final hidden representation at position t. This d-dimensional vector is projected back into the vocabulary space, often by multiplying it by the transpose of the embedding matrix E^T. When the same embedding matrix is reused in this way, the technique is known as weight tying.
- **Softmax**: The resulting logits are passed through a softmax function to produce a probability distribution over the next token:

$$\mathbf{p}_t = \text{softmax}(h_t E^T)$$

where $\mathbf{p}_t$ is the probability distribution over possible values of z_t given the prefix $z_{<t}$.

In general, this means that the model assigns a probability to each possible next token given the preceding trajectory prefix. In an NLP setting, where tokens correspond to words, this can be interpreted as the model estimating which word is most likely to come next in the sentence. The next token is then selected according to a decoding strategy such as greedy decoding, sampling, or beam search.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[3] Michael Janner, Qiyang Li, and Sergey Levine. Offline Reinforcement Learning as One Big Sequence Modeling Problem. *Advances in Neural Information Processing Systems*, 2021.

[4] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition. Online manuscript released January 6, 2026. https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf

[5] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[6] Diaeddin Alarnaouti, George Baryannis, Mauro Vallati. *Reformulation techniques for automated planning: a systematic review. The Knowledge Engineering Review*, 38, 2023.